

DESIGN AND ANALYSIS ALGORITHMS

LECTURE 7

Dynamic Programming (Quy hoạch động)

Review the Previous Lesson

Greedy method

- ❑ Greedy strategy, template and diagram
 - ❑ Examples
 - Minimal Spanning Tree: Prim, Kruskal
 - Shortest path in graph: Dijkstra
 - Lossless data compression: Huffman code
 - ❑ Advantage and disadvantage of the method
 - Optimal solution
 - Time complexity
-

Lecture outline

- ❑ Method introduction
- ❑ Typical steps to design algorithm
- ❑ Algorithm analysis
- ❑ Examples
- ❑ Caution: Common mistakes
- ❑ Exercise

Reference link and book chapter

Prof. Xukai Zou, Computer Science Dept.,
Indiana University Purdue University Indianapolis
<http://cs.iupui.edu/~xzou/>

Prof. Pasi Fränti, School of Computing,
University of Eastern Finland
<http://cs.uef.fi/pages/franti/asa/>

Anany's book, chapter 8

Introduction

- ❑ Dynamic programming (DP) – Quy hoạch động (QHĐ)
 - Invented by Richard Bellman (USA), 1950s
 - “Programming” ~ “Planning” not “Computer programming”
 - Tool of applied mathematics $\Rightarrow$ algorithm design method
- DP is **a simple technique** but it can be **difficult to master**.
Mastering the method would greatly increase your skill.

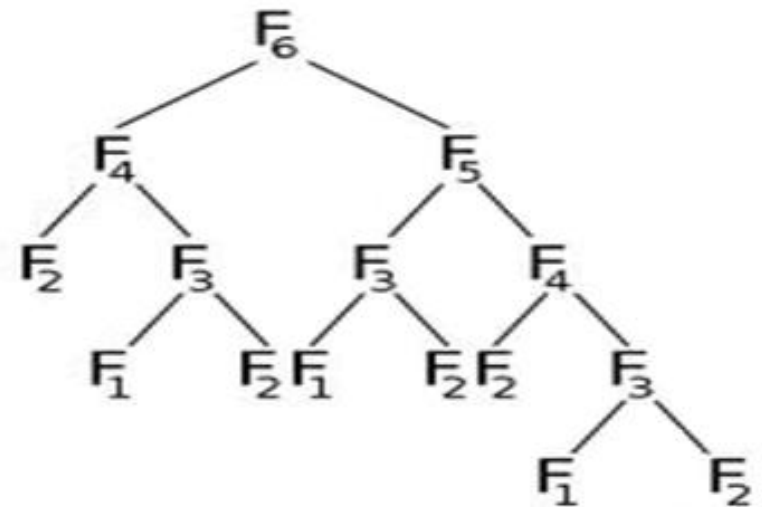
Introduction

□ Dynamic Programming Strategy

- Look back Fibonacci problem

Calculate the n^{th} Fibonacci

```
F(n) ≡  
  if (n=1 || 2)  
    F = 1;  
  else  
    F = F(n-1) + F(n-2);  
End.
```



- Limitation: Overlap subproblems, $O(2^n)$
- Solution:
 - Solve subproblems, record the solutions in a table
 - Combine the subproblem results for larger problems

$F_1 ; F_2 \rightarrow F_3 \rightarrow \dots \rightarrow F_n$

Introduction

❑ Dynamic Programming Strategy

- DP method solving problem with follow principle:
 - Solve smaller (overlapping) subproblems (one time).
 - Record solution in a table.
 - Combine recorded results to get the solution.

❑ Applicability of DP

- Optimization problem: find a solution with optimal (max/min) value.
An optimal solution, not **the** optimal solution.
- Recursive and overlapping sub-problems.

Introduction

□ DP **versus** D&C and Recursion

■ D&C and Recursion:

- Coi như các bài toán con đã có lời giải, qui bài toán lớn về các bài toán nhỏ hơn, cho tới khi gặp trường hợp suy biến.
- Lời giải cho cùng 1 bài toán có thể bị lặp lại.
- Phương thức Top-Down

■ DP:

- Giải trước các bài toán con rồi giải bài toán lớn dần.
- Kết quả của bài toán con (tối ưu), được lưu để tính kết quả bài toán lớn hơn, chỉ giải 1 lần.
- Phương thức Bottom-Up

Design algorithm DP

- ✓ An example
 - ✓ Main elements
 - ✓ Steps to implement
 - ✓ Analysis
-

Design algorithm DP – an example

□ Counting Combination Problem (đếm số tổ hợp)

- How many ways to choose r items from n given items.
- Divide and conquer: Choose the first item and not choose

$$C(r,n) = C(r-1,n-1) + C(r,n-1)$$

- Algorithm

```
Choose(r,n) ≡  
    if (r=0||n=r) return (1);  
    else  
        return (Choose(r-1,n-1) + Choose(r,n-1));
```

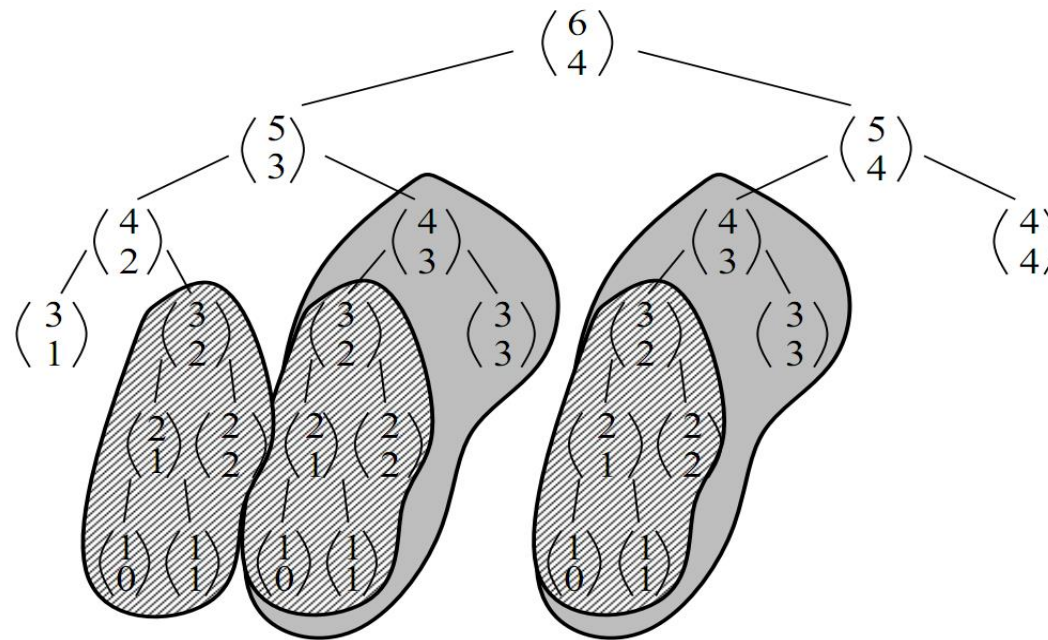
End.

- Time complexity: $T(n)=2T(n-1)+1 = O(2^n)$

Design algorithm DP – an example

Counting Combination Problem

- The problem: the algorithm solves the same subproblems over and over again! - overlapping subproblems



Design algorithm DP – an example

□ Counting Combination Problem

■ A better approach

- Use a table $C(i,j)$ to hold result subproblem choose i from j items

$$C(0,j) = 1; C(i,i) = 1$$

$$C(i,j) = C(i-1,j-1) + C(i,j-1) \quad i=1..r; j=1..n; i \leq j$$

- The solution $C(r,n)$ – value at bottom-right

- Illustration C with $r=5, n=10$

	0	1	2	3	4	5	6	7	8	9	10	n=10
0	1	1	1	1	1	1	1	1	1	1	1	
1		1	2	3	4	5	6	7	8	9	10	
2			1	3	6	10	15	21	28	36	45	
3				1	4	10	20	35	56	84	120	
4					1	5	15	35	70	126	210	
5						1	6	21	56	126	252	

r=5

Design algorithm DP – an example

□ Counting Combination Problem – better approach

■ Algorithm:

Choose(r, n) $\equiv$

```
for i = 1..r C[i,i]=1;
```

```
for i = 1..n-r C[0,i]=1;
```

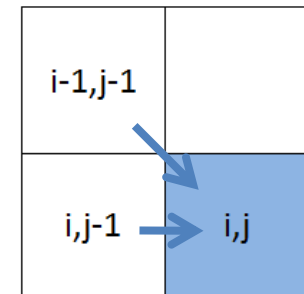
```
for i = 1..r
```

```
    for j = i+1..n-r+i
```

```
        C[i,j]=C[i-1,j-1]+C[i,j-1];
```

```
return (C[r,n]);
```

End.



■ Complexity:

- Time: $T(r,n)=(r+1)*(n-r+1)=nr+n-r^2+1 \leq n(r+1)+1=O(n^2) \ll O(2^n)$
- Space: Worst case $O(nr)$. Best case $O(r)$.

Design algorithm DP – main idea

- ❑ Main idea of DP algorithm
 - Record the results of subproblems, so that we do not have to re-compute them when needed later.
 - ❑ Two ways to store results of subproblems
 - Tabulation (Bảng phương án)
 - Starting from the smallest subproblem to larger, each once.
 - Bottom-up approaches.
 - Memoization (Ghi nhớ)
 - Starting from the original problem to smaller by recurrence, store the value each time subproblem solved.
 - Combine strengths of top-down and bottom-up approaches.
-

Design algorithm DP - elements

□ Basic elements in designing DP-tabulation

- **Recurrence** relation formular (Công thức truy hồi/QHĐ)
 - **Initial conditions** (Cơ sở QHĐ)
 - **Result table** (Bảng kết quả/Bảng phương án)
 - **Optimal solution** (Kết quả tối ưu của bài toán)
 - **Back-trace** to indicate the solution (Truy vết tìm nghiệm)
-

Design algorithm DP - elements

□ Công thức truy hồi (Recurrence)

- Công thức xác định lời giải bài toán lớn hơn thông qua các bài toán nhỏ hơn
- Chia bài toán thành N giai đoạn, tối ưu dần từng giai đoạn từ $0, 1, \dots, N$.
- Giai đoạn thứ i : Gọi $S(i)$ là hàm số xác định giá trị tối ưu (max/min) cần xác định. Khi đã biết các giá trị tối ưu (max/min) $S(j)$ của các giai đoạn $j=0, \dots, i-1$ thì:

$S(i) = \text{Min} \{S(j) \text{ kết hợp với Giá trị nhỏ nhất từ } j \text{ đến } i\}$ với $j=0, 1, 2, \dots, i-1$

Hoặc

$S(i) = \text{Max} \{S(j) \text{ kết hợp với Giá trị lớn nhất từ } j \text{ đến } i\}$ với $j=0, 1, 2, \dots, i-1$

$S(N) \leftarrow S(k) \forall k < N \leftarrow S(j) \forall j < k \leftarrow \dots \leftarrow S(2), S(1), S(0)$

Design algorithm DP - elements

□ Cơ sở QHĐ (Initial conditions)

$S(N) \leftarrow S(k) \forall k < N \leftarrow S(j) \forall j < k \leftarrow \dots \leftarrow S(2), S(1), S(0)$

- Là các giá trị tối ưu có thể tính được ngay $S(2), S(1), S(0)$
- Thường tương ứng với trường hợp suy biến của thuật toán đệ quy

Design algorithm DP - elements

□ Bảng phương án (Result table)

- Bảng lưu tất cả các giá trị tối ưu $F(i)$ ($i=0,1,\dots,N$) của các bài toán con.
- Có thể là 1 chiều (mảng), 2 chiều (ma trận) hoặc nhiều hơn
- Cần chú ý cách phối hợp kết quả của các bài toán để tìm kết quả cho bài toán lớn hơn (dựa vào công thức truy hồi)

Design algorithm DP - elements

❑ Kết quả tối ưu của bài toán (Optimal solution)

- Nằm trong bảng phương án
- Vị trí: Tùy thuộc bài toán

❑ Truy vết tìm nghiệm (Back-trace)

- Dựa vào bảng phương án để tìm ra các thành phần tham gia nghiệm.
 - Điểm bắt đầu của quá trình truy vết: ô chứa giá trị tối ưu, kết quả của bài toán.
 - Điểm kết thúc của quá trình truy vết: tùy thuộc bài toán, thường nằm ở những dòng, những cột đầu tiên của bảng phương án.
-

Design algorithm DP - implement

□ Các bước xây dựng thuật toán

Thiết kế thuật toán

1. Nhận dạng bài toán giải bằng QHĐ
2. Xây dựng công thức truy hồi
3. Xác định cơ sở QHĐ

Triển khai thuật toán

4. Dựng bảng phương án
 5. Tìm kết quả tối ưu – nghiệm của bài toán
 6. Truy vết liệt kê thành phần nghiệm
-

Design algorithm DP - implement

❑ Nhận dạng bài toán giải bằng QHĐ

- Dựa vào dấu hiệu nhận dạng bài toán giải bằng QHĐ
- Chú ý: không phải bài toán nào giải theo QHĐ cũng tốt hơn đệ qui, chẳng hạn như bài toán “Xếp ba lô 0-1” với số liệu lớn.

❑ Xây dựng công thức truy hồi

- Đưa bài toán về 1 dạng cơ bản, và triển khai ý tưởng của dạng bài toán đó để nhanh chóng nhận ra hướng thiết lập công thức truy hồi.
- Đây là bước khó nhất và cũng quan trọng nhất trong toàn bộ quá trình thiết kế thuật toán cho bài toán.

❑ Xác định cơ sở QHĐ

- Dựa vào công thức truy hồi để nhận ra các bài toán cơ sở.
- Dựa vào ý nghĩa của công thức truy hồi để thiết lập giá trị cho cơ sở.

Design algorithm DP - implement

□ **Dựng bảng phương án**

- Dựa vào công thức truy hồi để tính giá trị các ô trong bảng phương án.
- Chú ý: bảng phương án có thể 1 chiều, 2 chiều hoặc nhiều hơn

□ **Tìm kết quả tối ưu**

- Xác định vị trí chứa kết quả tối ưu của bài toán trên bảng phương án.
- Chú ý: ngoài kết quả tối ưu, ô chứa kết quả tối ưu còn là điểm bắt đầu cho quá trình truy vết tìm nghiệm => lưu tọa độ của ô đó.

□ **Truy vết liệt kê thành phần nghiệm**

- Từ điểm bắt đầu là vị trí chứa kết quả tối ưu
- Truy ngược lại về điểm bắt đầu của nghiệm: có thể là những ô đầu tiên trong bảng phương án (bài toán cơ sở), có thể là ô của bảng phương án đạt giá trị đầu.

Design algorithm DP - analysis

□ Chứng minh tính đúng - Proof the correctness

- Giai đoạn xây dựng bảng phương án

```
for i = 1..r C[i,i]=1;  
  for i = 1..n-r C[0,i]=1;  
    for i = 1..r  
      for j = i+1..n-r+i  
        C[i,j]=C[i-1,j-1]+C[i,j-1];
```

- Sử dụng quy nạp hoặc bất biến vòng lặp
- Giai đoạn truy vết tìm nghiệm
 - Sử dụng quy nạp
 - Sử dụng bất biến vòng lặp

Design algorithm DP - analysis

□ Đánh giá độ phức tạp – Analyze the complexity

■ Độ phức tạp thời gian

- Thời gian tính toán, xác định giá trị các phần tử trong bảng phương án, phụ thuộc vào số chiều của bảng phương án.

$$T(n) = O(n^k) \quad - k \text{ số chiều của bảng phương án}$$

- Thời gian truy vết tìm nghiệm: Độ phức tạp thuật toán quay lui

■ Độ phức tạp không gian

- Kích thước bảng phương án – thường lớn
- Giảm kích thước bảng phương án trong triển khai thuật toán

Examples

-
- ✓ Longest increasing subsequence
 - ✓ Knapsack
 - ✓ Path in numeric table
-

Longest increasing subsequence

□ Bài toán dãy con tăng dài nhất

- Tìm dãy con dài nhất của một dãy đã cho. Các phần tử có thể không liên tiếp.
Ví dụ: Dãy 2; 5; 4; 6; 3; 8; 9; 7
=> 2; 5; 6; 8; 9
- Phân tích:
 - Input: (a, n)
 - Output: m – số lớn nhất các phần tử của dãy theo thứ tự tăng dần
 - Hàm tối ưu $L(i)$: Độ dài dãy con đơn điệu tăng dài nhất đến phần tử i
Độ dài dãy con dài nhất đến j cộng 1 khi ghép thêm a_i vào sau, với $j < i$, và $a_j < a_i$
 - Công thức truy hồi: $L(i) = \max\{L(j)\} + 1$ với $j < i$, $a_j < a_i$
 - Cơ sở QHĐ: $L(0) = 0$; $L(1) = 1$

Longest increasing subsequence

□ Bảng phương án

- Bảng phương án: Bảng 1 chiều L

i	0	1	2	3	4	5	6	7	8
a_i		2	5	4	6	3	8	9	7
$L(i)$	0	1	2	2	3	2	4	5	4

- Giá trị tối ưu: $\max(L(i))_{i=1..n} = 5$
- Vậy dãy con đơn điệu tăng dài nhất có độ dài là 5
- Vấn đề đặt ra: Dãy nào?

Giá trị tối ưu, bảng phương án $\Rightarrow$ dãy: truy vết tìm nghiệm

Longest increasing subsequence

❑ Truy vết tìm nghiệm

- Bảng phương án: Bảng 1 chiều L

i	0	1	2	3	4	5	6	7	8
a_i		2	5	4	6	3	8	9	7
$L(i)$	0	1	2	2	3	2	4	5	4

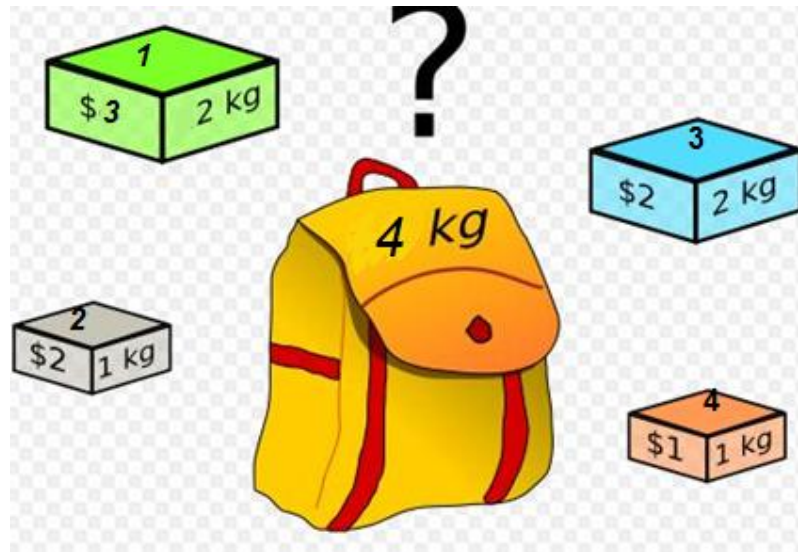


- Bắt đầu từ vị trí tối ưu, $L(i)$ đạt max.
- Tại mỗi bước lùi về trước: $a_j, L(j)$, mà $a_j < a_i$ và $L(j) = L(i) - 1$.
- Kết thúc khi đến đầu dãy, $L(j) = 0$
- Chú ý: Có thể có nhiều giá trị cùng thỏa mãn có thể chọn 1 hoặc tất cả.

0-1 knapsack

□ Bài toán xếp balo 0-1

- N đồ vật với trọng lượng và giá trị (p_i, w_i) . Cách cho các vật vào balo có trọng lượng P để đạt giá trị cao nhất. Mỗi vật chỉ được chọn 1 lần (0-1).
- Input: $n, (p_i, w_i) i=1..n, P$
- Output: W = tổng giá trị lớn nhất các đồ vật cho vào balo



0-1 knapsack

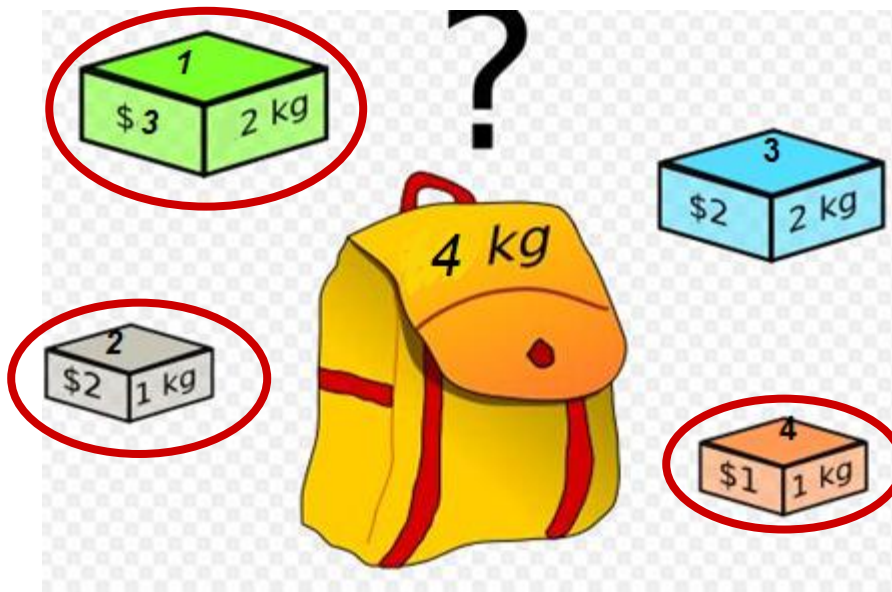
□ Công thức truy hồi

- Hàm tối ưu $C(i,j)$: Giá trị lớn nhất khi chọn các vật từ 1 đến i với trọng lượng balo j ($i=0..n, j=0..W$).
 - Nếu không chọn đồ vật thứ i thì: $C(i,j) = C(i-1,j)$
 - Nếu có chọn đồ vật thứ i thì $C(i,j) = C(i-1,j-p_i) + w_i$ (điều kiện $p_i \leq j$)
- Công thức truy hồi
$$C(i,j) = \max(C(i-1,j), C(i-1,j-p_i) + w_i)$$
- Cơ sở QHD
$$C(i,0) = 0$$
$$C(0,j) = 0$$

0-1 knapsack

□ Bảng phương án

- Bảng hai chiều lưu các giá trị $C(i,j)$: $C(i,j) = \max(C(i-1,j), C(i-1,j-p_i)+w_i)$



	0	1	2	3	4
0	0	0	0	0	0
1	0	0	3	3	3
2	0	2	3	5	5
3	0	2	3	5	5
4	0	2	3	5	6

- Giá trị tối ưu: $C(n,W) = 6$
- Những vật nào sẽ cho vào balo? $\Rightarrow$ Truy vết tìm nghiệm

0-1 knapsack

Truy vết tìm nghiệm

- Bảng hai chiều lưu các giá trị $C(i,j)$: $C(i,j) = \max(C(i-1,j), C(i-1,j-p_i)+w_i)$

	0	1	2	3	4
0	0	0	0	0	0
1	0	0	3	3	3
2	0	2	3	5	5
3	0	2	3	5	5
4	0	2	3	5	6

i	p_i	w_i	Chọn
0			
1	2	3	✓
2	1	2	✓
3	2	2	x
4	1	1	✓

- Bắt đầu từ ô $C(n,W)$
 - Nếu $C(i,j) = C(i-1,j)$ vật thứ i không được chọn và xét $C(i-1,j)$
 - Ngược lại $C(i,j) = C(i-1,j-p_i)+w_i$ vật thứ i được chọn và xét $C(i-1,j-p_i)$
- Kết thúc khi tới hàng 0 hoặc cột 0

Path in numeric table

□ Bài toán đường đi trong bảng số

- Cho bảng số dương kích thước $m \times n$. Từ 1 ô (i, j) có thể đi tới các ô phía bên phải $(i-1, j+1)$, $(i, j+1)$, $(i+1, j+1)$. Tìm cách đi từ bên trái sang ô (m, n) sao cho tổng giá trị các ô trên đường đi là lớn nhất.

- Input: Ma trận (a, m, n)

- Output:

- Tổng giá trị lớn nhất các ô đi từ $(*, 1)$ đến (m, n)
- Các ô trên đường đi.

9	9	9	2	6	1	6	1
4	5	2	1	4	5	1	2
2	3	9	8	1	3	4	3
5	7	2	1	9	8	1	4
9	8	2	1	4	7	6	5
3	7	1	3	6	8	1	6
1	2	3	4	5	6	7	

- Ví dụ:

Đường đi: từ $(1,1)$ đến $(6,7)$, tổng 48. Dài nhất?

Path in numeric table

□ Công thức truy hồi

- Hàm tối ưu: $C(i,j)$ là tổng giá trị các ô trên đường đi tới ô (i,j)
- Để đến được ô (i,j) , có thể từ 1 trong 3 ô $(i-1,j-1)$, $(i,j-1)$, $(i+1,j-1)$

$$C(i,j) = \max\{C(i-1,j-1), C(i,j-1), C(i+1,j-1)\} + a_{ij}$$

Chú ý là các ô được xét phải nằm trong bảng số.

□ Cơ sở QHĐ

$$C(i,0) = C(0,j) = 0$$

9	9	9	2	6	1	6	1
4	5	2	1	4	5	1	2
2	3	9	8	1	3	4	3
5	7	2	1	9	8	1	4
9	8	2	1	4	7	6	5
3	7	1	3	6	8	1	6
1	2	3	4	5	6	7	

Path in numeric table

❑ Nghiệm và truy vết tìm nghiệm

- Nghiệm $C(m,n)$
- Truy vết từ ô $C(m,n)$ lùi cột trái khi tới cột 0

9	9	9	2	6	1	6
4	5	2	1	4	5	1
2	3	9	8	1	3	4
5	7	2	1	9	8	1
9	8	2	1	4	7	6
3	7	1	3	6	8	1

Bảng số a_{mn}

	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	9	18	27	29	35	38	48
2	0	4	14	20	28	37	42	46
3	0	2	8	25	33	34	45	54
4	0	5	16	19	26	42	50	51
5	0	9	17	19	20	30	49	56
6	0	3	16	18	22	28	38	50

Bảng phương án C_{mn}

Path in numeric table

□ Lược đồ chương trình

- Xây dựng bảng phương án

//Cơ sở QHĐ

```
for (i=0...m) C[i][0] = 0
```

```
for (j=0...n) C[0][j] = 0
```

//Công thức truy hồi

```
for (i=1...m)
```

```
    for (j=1...n)
```

```
        C[i][j] = max(C[i-1][j-1],  
                      C[i][j-1],  
                      C[i+1][j-1]) + A[i][j]
```

Path in numeric table

□ Lược đồ chương trình

- Truy vết tìm nghiệm

```
i=m; j=n;  
while (j>1)  
    print (i, j)  
    if (i>1)  
        k = C[i][j]-A[i][j];  
        if (C[i-1][j-1]=k) {i--; j--;}  
        else if (C[i][j-1]=k) j--;  
        else i--;  
    else j--;  
endw;
```

	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	9	18	27	29	35	38	48
2	0	4	14	20	28	37	42	46
3	0	2	8	25	33	34	45	54
4	0	5	16	19	26	42	50	51
5	0	9	17	19	20	30	49	56
6	0	3	16	18	22	28	38	50

Common mistakes

❑ Lỗi thường gặp khi thiết kế thuật toán

- Nhận dạng không đúng bài toán
 - Xác định sai công thức truy hồi
 - Xác định sai giá trị tối ưu cần tính
 - Nhầm lẫn giữa các cách kết hợp bài toán con
 - Xác định sai ý nghĩa của bảng phương án
 - Xác định sai hoặc thiếu cơ sở QHĐ
 - Xác định sai kết quả tối ưu
 - Xác định sai điều kiện dừng khi truy vết
-

Common mistakes

❑ Lỗi thường gặp khi triển khai thuật toán

- Khai báo kích thước bảng phương án nhỏ hơn yêu cầu
 - Bảng phương án có thể có thêm viền (hàng số 0, cột số 0)
 - Tính sai giá trị phần tử bảng phương án
 - Tùy vào công thức truy hồi mà mỗi phần tử được tính từ phần tử khác
 - Xét ô ở ngoài bảng
 - Khi tính bảng phương án hoặc truy vết, cần chú ý các chỉ số để đảm bảo phần tử ở trong bảng.
-

Applications

❑ Một số bài toán giải bằng QHĐ

<https://www.geeksforgeeks.org/dynamic-programming/>

Basic Problems: 106

Intermediate Problems: 108

Hard Problems: 49

Summary

- ❑ **Nhận dạng bài toán giải bằng QHĐ**
 - Bài toán có thể giải bằng đệ quy và tìm lời giải tối ưu.
 - ❑ **Khi nào sử dụng được phương pháp QHĐ?**
 - Trả lời: Nếu tìm được công thức truy hồi.
Tìm công thức truy hồi là việc quan trọng nhất.
 - Hơn nữa: Đủ không gian bộ nhớ cho bảng phương án.
 - ❑ **Các bước thiết kế thuật toán**
 - Xác định hàm mục tiêu cần tối ưu
 - Xây dựng công thức truy hồi, bảng phương án
 - Xác định nghiệm.
 - ❑ **Lỗi khi thiết kế và triển khai thuật toán**
-

Exercises

- ❑ Anany's book chapter 8
- ❑ Other examples or applications of greedy method
- ❑ More detail in [Hw7_DynamicProgramming.doc](#)